

CS-3510-B Exam 2

Elijah Martin McCauley

TOTAL POINTS

112 / 115

QUESTION 1

Problem 1 18 pts

1.1 Part (i) 3 / 3

- ✓ - 0 pts Correct (True)
- 3 pts Incorrect

1.2 Part (ii) 3 / 3

- ✓ - 0 pts Correct (False)
- 3 pts Incorrect

1.3 Part (iii) 0 / 3

- 0 pts Correct (False)
- ✓ - 3 pts Incorrect

1.4 Part (iv) 3 / 3

- ✓ - 0 pts Correct (False)
- 3 pts Incorrect

1.5 Part (v) 3 / 3

- ✓ - 0 pts Correct (False)
- 3 pts Incorrect

1.6 Part (vi) 3 / 3

- ✓ - 0 pts Correct (False)
- 3 pts Incorrect

QUESTION 2

2 Problem 2 18 / 18

✓ - 0 pts Correct

- 2 pts Didn't address equals case
- 3 pts Minor Error (Correct Algorithm, but slightly flawed details)
- 5 pts Incorrect/ Missing Runtime Analysis
- 7 pts Major Error/ Quite flawed details
- 8 pts Non $O(1)$ Solutions
- 13 pts Incorrect Algorithm, but compatible runtime
- 18 pts Missing answer

QUESTION 3

3 Problem 3 20 / 20

- ✓ - 0 pts Correct (Residual Network and Max Flow = 6)
- 2 pts One to two edges incorrectly drawn/missing
- 5 pts Three to four edges incorrectly drawn/missing
- 7 pts Five to six edges incorrectly drawn/missing
- 7 pts Edges reversed in Residual Graph
- 10 pts More than six edges incorrect/missing
- 15 pts Incorrect Residual Graph
- 5 pts Incorrect Max Flow

QUESTION 4

Problem 4 22 pts

4.1 Part (i) 4 / 4

✓ - 0 pts Correct (5)

- 4 pts Incorrect

4.2 Part (ii) 4 / 4

✓ - 0 pts Correct (B), (E)

- 2 pts Either B or E missing/ Added more vertices than B & E

- 4 pts Incorrect/ Missing Answer

4.3 Part (iii) 4 / 4

✓ - 0 pts Correct (CDFJ or any other order)

- 2 pts Minor Error/ Added more vertices than CDFJ

- 4 pts Incorrect/ Missing Answer

4.4 Part (iv) 4 / 4

✓ - 0 pts Correct

(A: 16, B: 18, C: 9, D: 8, E: 20, F: 7, G: 14, H: 15, I: 13, J: 6)

- 1 pts One to two post numbers incorrect

- 2 pts Three to four post numbers incorrect

- 3 pts More than four post numbers incorrect, but a few correct

- 4 pts Incorrect/ Missing Answer

4.5 Part (v) 3 / 3

✓ - 0 pts Correct (False)

- 3 pts Incorrect

4.6 Part (vi) 3 / 3

✓ - 0 pts Correct (False)

- 3 pts Incorrect

QUESTION 5

5 Problem 5 22 / 22

✓ - 0 pts Correct

- 5 pts Minor Error (Correct Algorithm, but slightly flawed details)

- 5 pts Incorrect/ Missing Runtime Analysis

- 7 pts Incorrect algorithm, but partially correct approach

- 8 pts Major Error/ Quite flawed details

- 10 pts Non-linear time solution

- 17 pts Incorrect Algorithm, but compatible runtime

- 22 pts Missing answer or totally incorrect

QUESTION 6

6 Problem 6 (Extra Credit) 15 / 15

✓ - 0 pts Correct (Yes), with valid reasoning

- 5 pts Answered No/ Depends

- 3 pts Case 1 Explanation Missing/ Incorrect

- 3 pts Case 2 Explanation Missing/ Incorrect

- 4 pts Case 3 Explanation Missing/ Incorrect

- 15 pts Missing Answer

GTID: 907684653
NAME: Elijah McCauley

Georgia Institute of Technology

Spring 2023

CS 3510B – Design & Analysis of Algorithms
Exam 2

February 23, 2023

TIME ALLOWED: 75 MINS

Name: Elijah McCauley
GTID: 907684653
GT Username: emccauley6

INSTRUCTIONS TO CANDIDATES

1. Please write your NAME and GTID clearly on all the pages.
2. This examination paper is worth **115 points** (100 + 15 extra credit points) comprising **SIX (6)** questions and TEN printed pages.
3. **ONLY** write on the front sheets of paper that are numbered. The backs will not be scanned.
4. Wherever a blank is provided, please write your answer in the given blank.
5. Calculators are **NOT** allowed.

I am in aware of and the accordance with Academic Honor Code of Georgia Tech and the Georgia Tech Code of Conduct. I'll use no external help on this test. Also, I have read all the instructions on this page.

Signature: 

Problem 1 (18 points; 3 points each)

Indicate whether the following statements are *true* or *false*.

- (i) Let T be some Minimum Spanning Tree of G . If we increase the weight of every edge in G by 50, then T is still a valid Minimum Spanning Tree.

Answer: T

- (ii) If you explore from a vertex in a source SCC, then you must discover all vertices in the graph.

Answer: F



- (iii) Assume we use Dijkstra's algorithm to compute some shortest path P between any two vertices $u, v \in V$ in a positive edge weighted graph. If we add 50 to the weight of every edge in the graph, then P is still a shortest path between u and v .

Answer: T

relative same

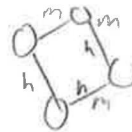
- (iv) In an undirected graph, the shortest path between any two nodes always lies on some Minimum Spanning Tree.

Answer: F



- (v) The Maximum Spanning Tree of a graph is always exactly the set of edges not in the Minimum Spanning Tree.

Answer: F



- (vi) The heaviest edge in the graph cannot belong to a minimum spanning tree.

Answer: F



Problem 2 (18 points)

You are given an undirected graph $G = (V, E)$ with positive weights, a starting vertex $u \in V$, and you have already run Dijkstra's Algorithm on the graph from source u to obtain an array $dist[]$, where for every $v \in V$, $dist[v]$ is the length of the shortest path from u to v . (Note: This time doesn't add to your algorithm in part (i)).

But then you are given some new edge $e' = (u, x)$ from the starting vertex to some other vertex $x \in V$, with weight $w(e') > 0$, which was not originally present in E .

We want to design an algorithm that checks if $dist[]$ remains correct on the new graph $G' = (V, E \cup \{e'\})$.

Input: Graph $G = (V, E)$, the edge weights w , $dist[]$, the new edge e' and its weight $w(e')$.

Output: True if $dist[]$ is the same on the new graph, and False otherwise.

- (i) (13 points) Design the most efficient algorithm (only the best runtime will receive full credit) by providing the pseudo-code **OR** explaining it in English. You may use any algorithm we discussed in class as a black box.

Hint: Running Dijkstra's algorithm again on the new graph, G' , is not the best approach.

We need to compare the old distances to each node, to the shortest distance to each node now that e' exists. There is actually only one thing we need to check to find out if this works, which is compare the new distance to x and the old distance to x . If the new edge from u to x has less weight than the old weight to get from u to x found with Dijkstra's, then we know that x 's weight will get updated to simply be $w(e')$ and that would change $dist[]$. So we return false. On next page.

we want to compare e to original $dist$ and e' results

Need to be better than Dijkstra's

If the new $w(e')$ is greater than the previously found shortest distance to x , then we can return true because if the shortest distance to x is unaffected by adding e , then it wouldn't affect the distance to any other node because if their distance is based on the distance to x , then it would still use the shorter distance previously found to get to x and then the same distance to get anywhere else. If $w(e') = \text{dist}[u, x]$ then it doesn't matter if we take the new or old path, so $\text{dist}[]$ doesn't change, so return true.

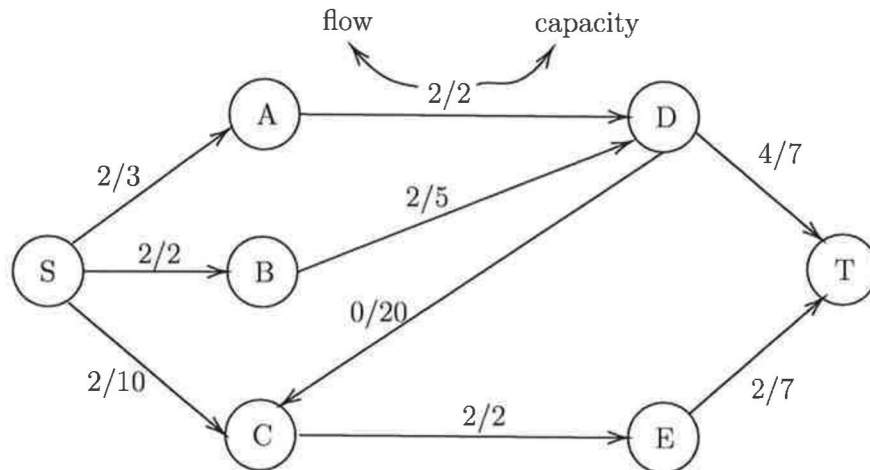
if $w(e') \geq \text{dist}[u, x]$
true
if $w(e') < \text{dist}[u, x]$
false

(ii) (5 points) Provide a brief explanation of your algorithm's runtime.

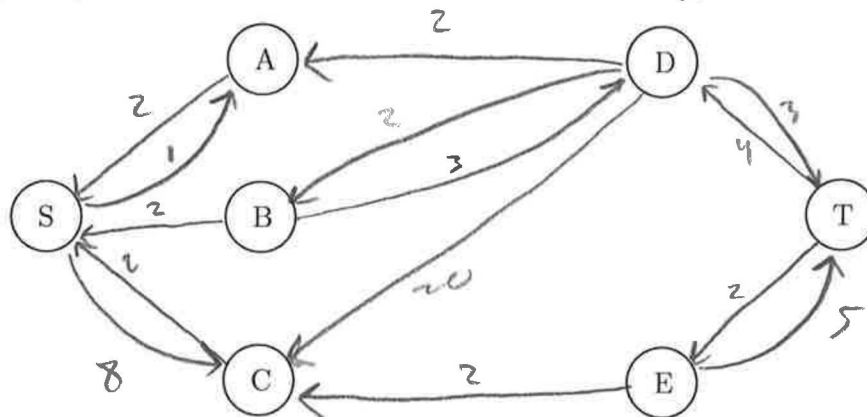
This algorithm would run in $O(1)$ time because we simply need to compare the value of $w(e')$ to the distance from u to x stored in the $\text{dist}[]$ table.

Problem 3 (20 points)

Consider the following flow network and feasible flow f from the source vertex S to the sink vertex T .



- (i) (15 points) Draw the residual network for the above flow f .

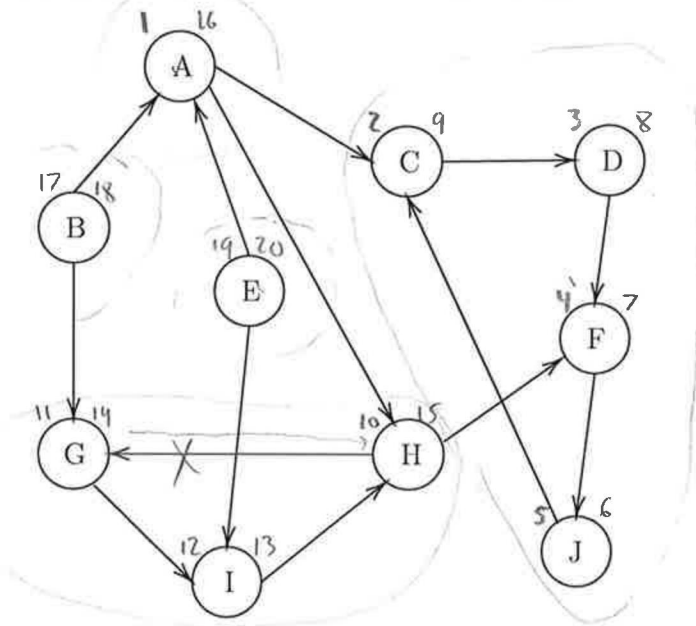


- (ii) (5 points) What is the maximum flow in the above graph? You do **not** need to justify your answer.

Answer: 6

Problem 4 (22 points)

Answer the questions using the graph G printed below. Write your answers on the line next to the question. You do not need to justify your answers.



- (i) (4 points) How many SCCs are in the graph G ?

Answer: 5

- (ii) (4 points) List all the source SCCs separated by commas (if multiple) in this graph G , as group of vertices (any ordering of vertices of an SCC is fine):

Answer: {A}, {B}, {E}, {G, H, I}, {C, D, F, J}

GTID: 909684653
NAME: Elijah McCawley

- (iii) (4 points) List all the sink SCCs separated by commas (if multiple) in this graph G , as group of vertices (any ordering of vertices of an SCC is fine):

Answer: {C D F J}

source, sink

- (iv) (4 points) Perform DFS with start vertex A and only list the **post numbers** for each vertex in this graph. Use alphabetical order of the vertices while performing DFS.

Answer:

A: 16
B: 18
C: 9
D: 8
E: 20
F: 7
G: 14
H: 15
I: 13
J: 6

- (v) (3 points) True or False: Deleting an edge from this graph G can only increase the number of SCCs by at most 1.

Answer: F

- (vi) (3 points) True or False: Reversing an edge from this graph G can only change the number of SCCs by at most 1.

Answer: F

Problem 5 (22 points)

You are given a weighted undirected graph $G = (V, E)$, weights given by $w(e) \forall e \in E$. Assume you already ran Kruskal's algorithm on graph G and obtained a minimum spanning tree, T . (Note: This time doesn't add to your algorithm in part (i)).

But, then you realize that you forgot to add an edge $e' = (u, v)$ where $u, v \in V$ with its weight as $w(e')$, which was not originally present in E (i.e. an edge doesn't exist between u and v in the original graph G). You have to design an algorithm that checks if the old tree T is still an MST on the new graph $G' = (V, E \cup \{e'\})$.

Input: Graph $G = (V, E)$, the edge weights w , old tree T , the new edge e' and its weight $w(e')$.

Output: True if the old tree T remains an MST for G' , and False otherwise.

- (i) (17 points) Design the most efficient algorithm (only the best runtime will receive full credit) by providing the pseudo-code **OR** and explaining it in English. You may use any algorithm we discussed in class as a black box.

Hint: Running Kruskal's algorithm again on the new graph, G' , is not the best approach.

Using our already found MST, we know that the edge e that we are adding forms a cycle since u and v have to already be in the tree. Because of this, for e' to be included in the MST, it needs to have a lower weight than one of the other edges in its cycle. → and therefore make old MST invalid
To check this, we run the SCC algorithm to output the nodes in each SCC. We can use this to get the nodes in the cycle that includes e' . Now we compare the weights of the other edges in the cycle with $w(e')$ and if any are higher than $w(e')$ we return false, else, e' has no effect on the MST and we return True.
↓
is the highest in the cycle and

GTID: 903684653
NAME: Elijah McCarty

(ii) (5 points) Provide a brief explanation of your algorithm's runtime.

The runtime of this algorithm is $O(V+E)$ because the initial Kruskal's call is not included, so we are simply calling SCC which is $O(V+E)$ and then comparing some weights which would take at worst $O(E)$ which would not effect the overall runtime.

Problem 6 (Extra Credit; 15 points)

Suppose we are given an undirected graph $G = (V, E)$ with positive weights $w(e)$ $\forall e \in E$. We are also given a minimum spanning tree T . Now suppose for every edge e we replace its edge weight by $w'(e)$ where:

$$w'(e) = \begin{cases} 1, & \text{if } w(e) \leq 100 \\ 2w(e), & \text{if } w(e) > 100 \end{cases} \quad (1)$$

Is the minimum spanning tree still guaranteed to be a minimum spanning tree for this new weighted graph with weights $w'(e)$ $\forall e \in E$? Explain your answer.

Hint: Explain what happens to each of these three cases: all edges in G are ≤ 100 , all edges in G are > 100 , and the graph has a mix of these edges to prove or disprove your claim.

Simply writing yes or no will receive zero points.

Answer: Yes

Case 1:

if all $w(e)$ are > 100 , then the mst is unaffected because all the weights are doubled and the relative weight remains the same.

Case 2:

if all $w(e) \leq 100$, then the mst is guaranteed to still be one because all the edges are 1, so every tree is an MST.

Case 3:

if the edges are mixed, then the old mst is guaranteed to still be one. Any edges greater than 100 that are in it will still be because they maintain their relative weight to the ones greater than them, and any $w(e) \leq 100$ go to 1 so any included are still optimal.